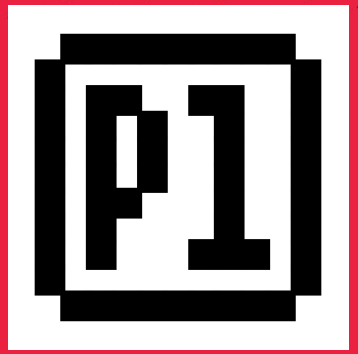
Contratos y testing



Martín René Vilugrón
mrvilugron@unrn.edu.ar

UNRN
Universidad Nacional
de Río Negro

10 / 2026

r33

Testing

UNRN Universidad Nacional de **Río Negro**

Objetivos

- Validar funcionalidad aislada
- Detectar errores tempranamente
- Facilitar refactorización
- Documentación del código
- Mejora de la calidad del código

{ y aprovechando la compilación separada }

Suma lenta

Reinventando la rueda de la suma, de la forma más ineficiente posible

¿Que podemos probar?

Algunos números importantes,

$$\{\pm 0, \pm 1, \pm 2, \pm 3, \pm 10\}$$

La estructura de una prueba

Puesta en marcha / Preparativos (setup)

```
int a = 5;
int b = 3;
```

Ejecución

```
int resultado = suma_lenta(a, b);
```

Verificación (aserciones)

```
assert(resultado == 8);
```

Limpieza (teardown)

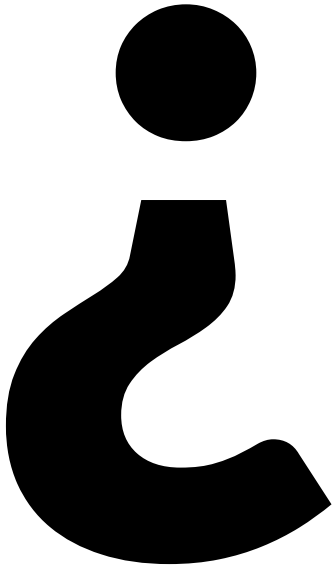
Acá no es estrictamente necesario, pero más adelante vamos a ver cuestiones que si necesitan de este paso.

```
void test_suma_positivo_positivo() {
    // Preparativos
    int a = 5;
    int b = 3;

    // Ejecución
    int resultado = suma(a, b);

    // Verificación
    assert(resultado == 8);

    // Teardown (no es necesario en este caso)
}
```



assert



assert.h

```
assert(condicion);
```

Si condición es falsa, el programa se abortará mostrando un mensaje de error.

Pero cuidado

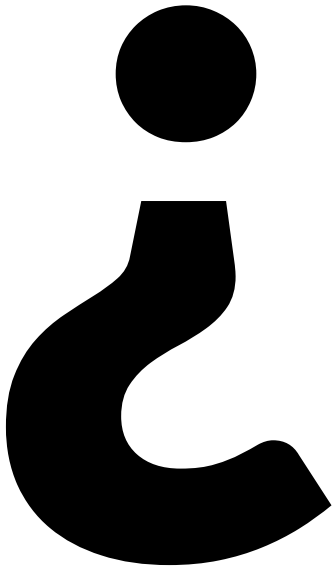
```
#define NDEBUG
#include <assert.h>
```

No está pensado para su uso en “Producción”

Un caso por función de prueba

¡A la terminal!





Como sabemos contra qué probar




¿Preguntas?

Hasta la próxima

UNRN Universidad Nacional de **Río Negro**

UNRN Universidad Nacional de **Río Negro**

@unrionegro

unrn.edu.ar

Probando software

UNRN Universidad Nacional de Río Negro

El camino hacia el desarrollo profesional de software

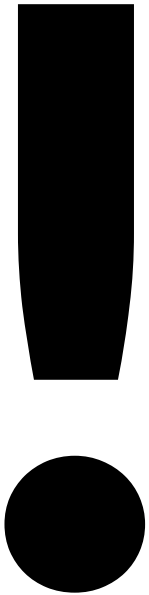
UNRN | Universidad Nacional de Río Negro

Y dejarnos de “codear”

Dando garantías de que el programa hace lo que se supone que debe



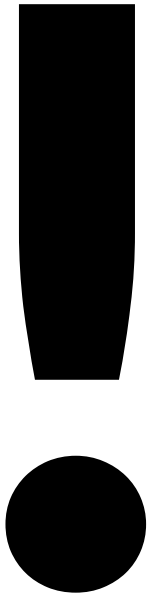
De forma *más* fehaciente



UNRN | Universidad Nacional de Río Negro



Este tema *es una disciplina completa*



Cultura general

UNRN Universidad Nacional de Río Negro

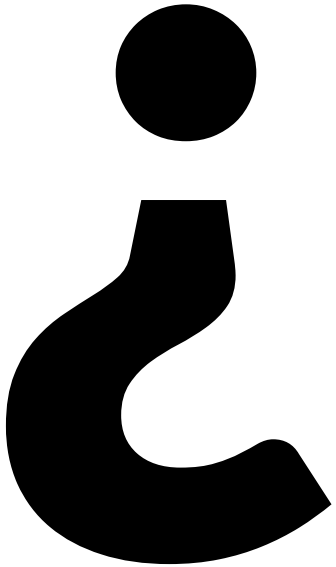
Oráculo

El juez imparcial, que contiene el resultado correcto y lo verifica con lo obtenido por el código.

(El código del test con los valores precalculados de los casos)

Sujeto Bajo Prueba (SUT)

Qué estamos probando, claramente delimitado y de alcance variable



Qué es exactamente un error



Error (Mistake / Equivocación)

Es la acción humana que produce un resultado incorrecto.

Defecto (Fault / Bug / Falta)

Es la manifestación estática del error en un artefacto de software (código fuente, especificación, modelo de datos).

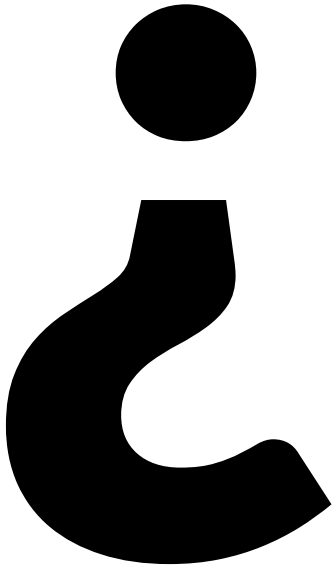
Fallo (Failure / Falla)

Es la desviación dinámica y observable del comportamiento del sistema respecto a su especificación funcional o no funcional.


El testing *dinámico*
no detecta errores
humanos
directamente

{ Su objetivo es provocar y exponer fallos observables }


¿Preguntas?



Cuándo una técnica de pruebas es superior a otra



{ No existe una técnica de pruebas universalmente "superior". }

No hay balas de plata



Ortogonalidad y Clases de defectos

Las técnicas son complementarias, no excluyentes. Una técnica es "superior" solo en relación con una clase específica de defecto.

Eficacia vs. eficiencia (ROI)

La superioridad se calcula como una función de rentabilidad.

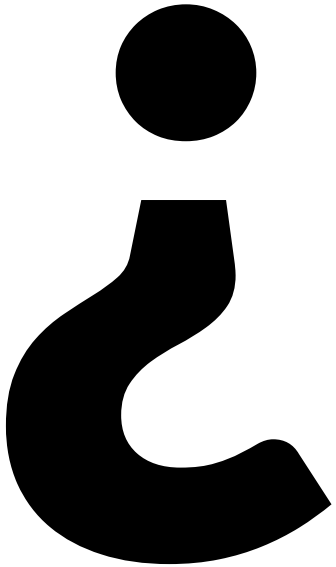
Criticidad y contexto del dominio

El dominio de aplicación dicta la técnica óptima. No es lo mismo probar SW de vuelo que algo comercial.

Rendimientos decrecientes y complementariedad

Es imperativo mencionar la Paradoja del Pesticida. La ejecución iterativa de una misma técnica *agota* su eficacia sobre un conjunto de código estático.


¿Preguntas?



Qué queremos obtener con las pruebas



Verificación y Validación (V&V)

La Verificación busca evaluar si el software cumple con su especificación.

La Validación determina si el sistema satisface las necesidades

Mitigación cuantitativa de riesgos

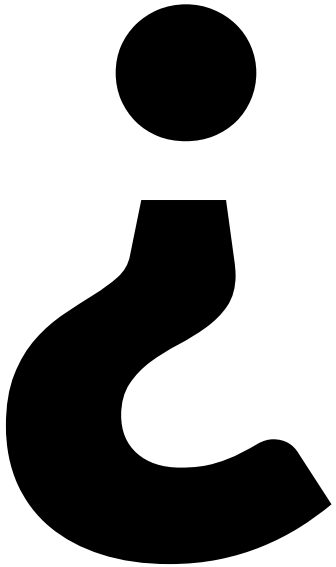
El testing es, en esencia, un ejercicio de gestión de la incertidumbre. Buscamos establecer un nivel de confianza estadísticamente válido sobre la calidad.

Control vs. Aseguramiento (QC vs. QA)

QC como la validación dinámica del SW, y ‘reactivo’

QA como la prevención ‘proactiva’ de defectos


¿Preguntas?



Cuando paramos de probar



Rendimientos decrecientes

Llega un punto donde encontrar el último "bug" cuesta más que lo que vale el software

Asíntota de Fallos

Dejamos de ver errores nuevos a pesar de probar más.

Riesgo aceptable

El software se libera cuando el riesgo residual no destruye la *reputación* ni la operación.


¿Preguntas?

Clasificaciones

UNRN Universidad Nacional de Río Negro

Clasificaciones

1

Por la técnica empleada y la información disponible

Caja negra

Orientado a la especificación. El SUT se modela como un sistema *opaco*; el código fuente, la arquitectura subyacente y los flujos de control internos son deliberadamente ignorados.

Caja blanca

Orientado a la implementación. Exige visibilidad total y acceso irrestricto al código fuente y la topología interna del software.

{ Nos vamos a centrar en las pruebas de caja blanca }


¿Preguntas?

Clasificaciones

2

Por el “nivel” de prueba

Unitario

Aislamiento de la unidad lógica mínima de código (funciones en C, métodos/clases en Java o Python).

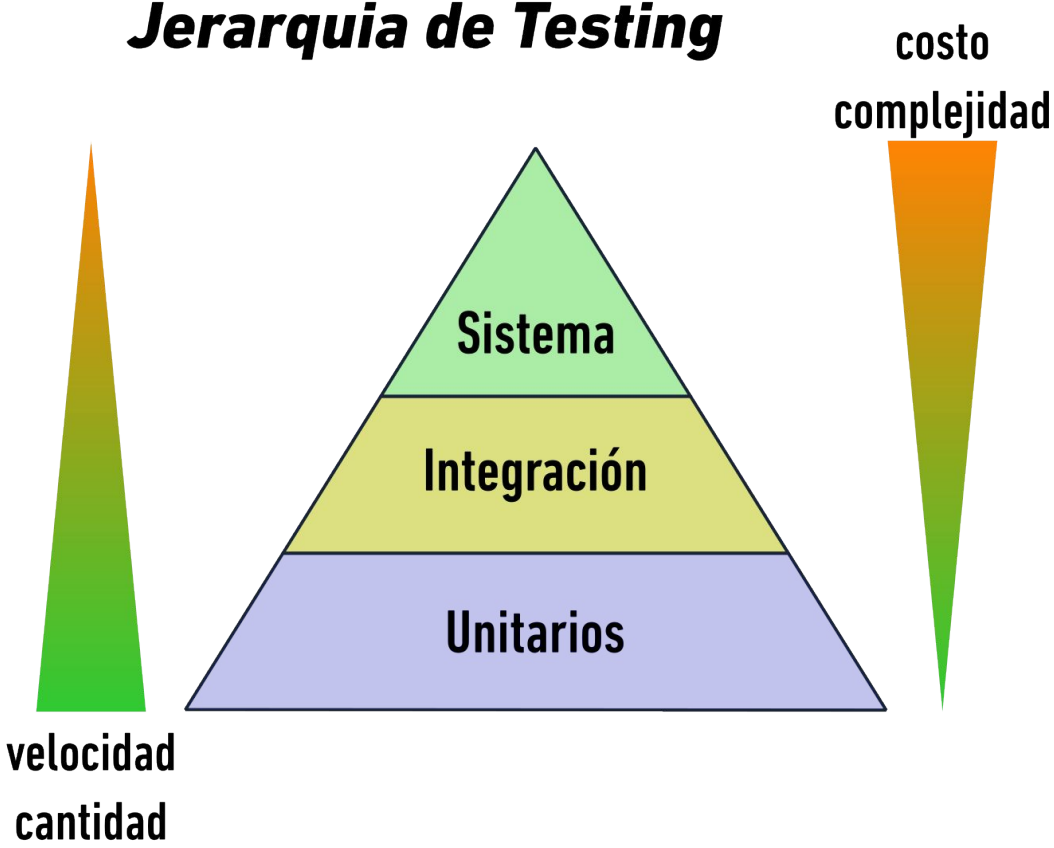
Integración

Evaluación estricta de las interfaces y el flujo de control/datos entre módulos ya probados unitariamente. Puede ser integración de componentes (interna) o integración de sistemas (APIs, microservicios, bases de datos).

Sistema

Validación del artefacto de software compilado e integrado en su totalidad, ejecutado en un entorno de pre-producción. Evalúa el cumplimiento holístico de los requerimientos.

Jerarquia de Testing

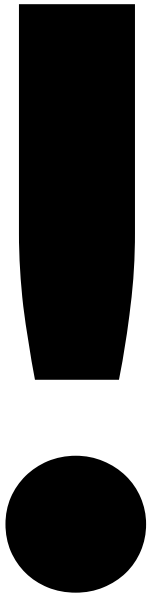


Nos vamos a centrar en el nivel unitario


¿Preguntas?



Testing es... *una disciplina completa*



¡Manos a la obra!

UNRN Universidad Nacional de **Río Negro**

Tests unitarios de caja blanca

UNRN Universidad Nacional de Río Negro

Usando

la estructura del código

la documentación

El oráculo

Detallando el contrato

Suma lenta de dígitos

Implementa una función que calcule la suma de los dígitos de un número entero positivo pasado como parámetro.

```
long sumaDigitos(int numero)
```

Detallando el contrato

Suma lenta de dígitos

Implementa una función que calcule la suma de los dígitos de un número entero positivo pasado como parámetro. Si el número es negativo, se retornará ERR_NUMERO_NEGATIVO.

```
long sumaDigitos(int numero)
```

Contratos

Es necesario reforzar

Pero las consignas son bastante vagas

Precondiciones

n is integer

n > 0

n < Long.MAX

Suma lenta de dígitos

Implementa una función que calcule la suma de los dígitos de un número entero positivo pasado como parámetro.

long sumaDigitos(int n)

Postcondiciones

ret > 0

ret = suma(digitos(n))

Suma lenta de dígitos

Implementa una función que calcule la suma de los dígitos de un número entero positivo pasado como parámetro.

long sumaDigitos(int n)

Pensar en precondiciones y postcondiciones ayuda a pensar la prueba


¿Preguntas?

Implementación

(sinó no hay caja blanca)

Algoritmo, sujeto bajo prueba (SUT)

```
public static long sumaDigitos(int numero) {
    if (numero <= 0) { // Nodo 1 (Defensa de Precondición)
       return ERR_NUMERO_NEGATIVO; // Nodo A
    }
    long suma = 0; // Nodo B
    while (numero > 0) { // Nodo 2
        suma = suma + (numero % 10); // Nodo C
        numero = numero / 10;
    }
    return suma; // Nodo D (Cumplimiento de Postcondición)
}
```

Caminos

“Fail fast”

1

Número negativo, obtenemos `ERR_NUMERO_NEGATIVO`

Probamos con algún valor negativo

```
public static long sumaDigitos(int numero) {
    if (numero <= 0) { // Nodo 1 (Defensa de Precondición)
        return ERR_NUMERO_NEGATIVO;
    }
    long suma = 0;
    while (numero > 0) { // Nodo 2
        suma = suma + (numero % 10);
        numero = numero / 10;
    }
    return suma;
}
```

Nodo 1 cubierto ✅

Caminos

Camino inviable

2

Bypass del lazo, inviable por la condición mutuamente excluyente.

`numero <= 0` y `numero > 0`

No afecta la cobertura (lógicamente inviable)

```
public static long sumaDigitos(int numero) {
    if (numero <= 0) { // Nodo 1 (Defensa de Precondición)
       return ERR_NUMERO_NEGATIVO;
    }
    long suma = 0;
    while (numero > 0) { // Nodo 2 (Lazo)
        suma = suma + (numero % 10);
        numero = numero / 10;
    }
    return suma;
}
```

Caminos

Ciclo normal

3

Es la ruta de procesamiento estándar. Ingresa al lazo (Decisión 2: Verdadera), ejecuta la mutación de estado (Nodo C), y reevalúa hasta satisfacer la condición de salida.

Probamos con el primer número y alguno más positivo

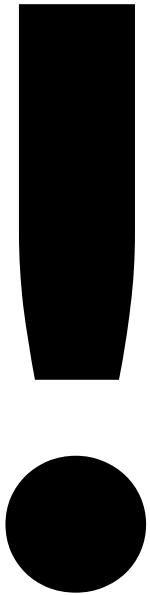
```
public static long sumaDigitos(int numero) {
    if (numero <= 0) { // Nodo 1 (Defensa de Precondición)
       return ERR_NUMERO_NEGATIVO;
    }
    long suma = 0;
    while (numero > 0) { // Nodo 2 (Lazo)
        suma = suma + (numero % 10);
        numero = numero / 10;
    }
    return suma;
}
```

Nodo 2 cubierto ✅

Cada rama probada aumenta la calidad del test



La especificación del problema juega fuerte.




¿Preguntas?

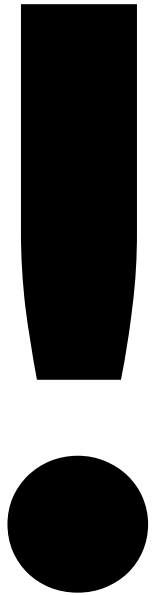
Lo que probamos *generalmente* no va a usar STDIN/STDOUT

{ Una buena razón por la cual `printf`/`Scanner` no van en las funciones }

El tener que crear el test nos ayuda a diseñar mejor



y, Es fundamental que falle, cuando tiene que fallar




¿Preguntas?

La anatomía de un caso de prueba

Arrange
Act
Assert

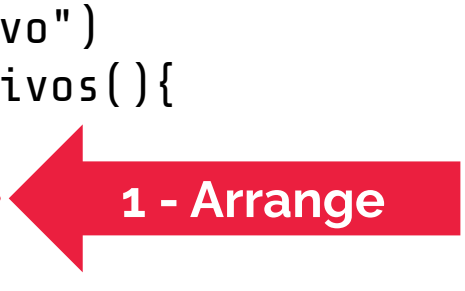
Preparar
Actuar
Afirmar

La estructura base

```
@Test
@DisplayName("Sumamo positivo")
public void test_suma_positivos(){
    int argumento1 = 10;
    int argumento2 = 20;
    int esperado = 30;

    int resultado = SumaApp.suma_lenta(argumento1, argumento2);

    assertEquals(esperado, resultado, "no coincide");
}
```







La estructura de un caso de prueba jUnit 5

```
@Test
@DisplayName("Sumamo positivo")
public void test_suma_positivos(){
    int argumento1 = 10;
    int argumento2 = 20;
    int esperado = 30;
    int resultado = SumaApp.suma_lenta(argumento1, argumento2);
    assertEquals(esperado, resultado, "no coincide");
}
```

Para saber qué caso es

La llamada a la unidad

Cuanto más explícito, mejor

Verificamos el resultado*

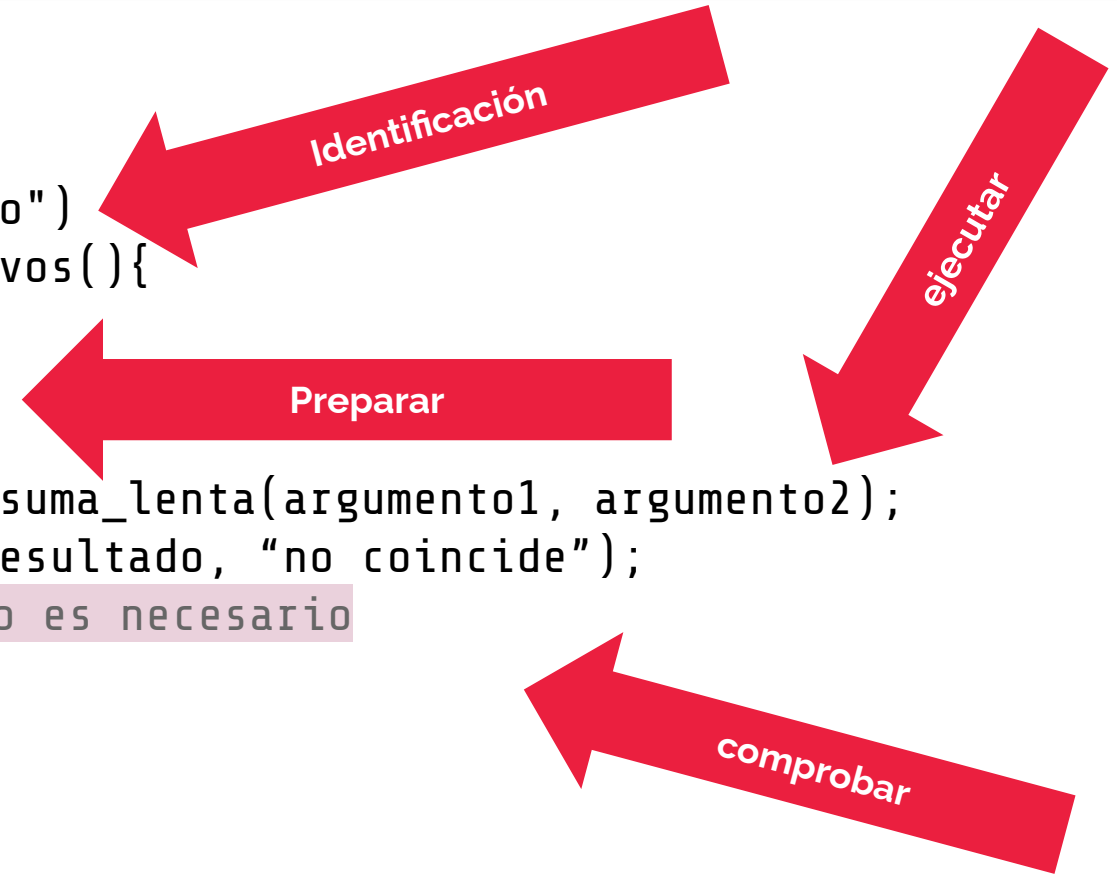
La estructura de un test individual debe ser:

preparar / ejecutar / comprobar / / *limpiar*

Con identificación y mensajes descriptivos

La estructura de un caso de prueba jUnit 5

```
@test
@DisplayName("Sumamo positivo")
public void test_suma_positivos(){
    int argumento1 = 10;
    int argumento2 = 20;
    int esperado = 30;
    int resultado = SumaApp.suma_lenta(argumento1, argumento2);
    assertEquals(esperado, resultado, “no coincide”);
    //limpiar en este caso no es necesario
}
```



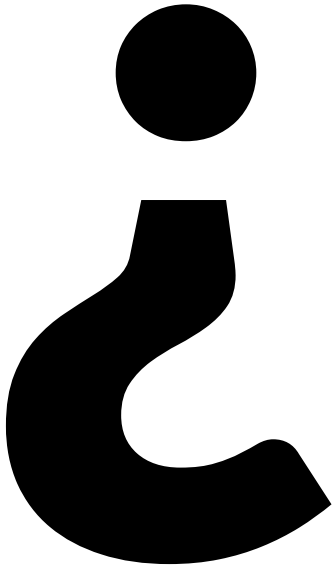
{ El código no suele ser más complicado que eso. }

{

El `message` *en sí* es opcional.

Describir cuál es la idea del caso de prueba no.

}



Que asserts tenemos



Tipos de assert

```
assertEquals(primitivo expected, primitivo actual, String message)

assertEquals(float expected, float actual, float delta, String message)

assertTrue(boolean condition, String message)

assertFalse(boolean condition, String message)

fail(String message)
```

Documentación

assertEquals

```
@Test
public void igualdadTest() {
    String esperado = "UNRN";
    String resultado = "UNRN";

    assertEquals(esperado, resultado, “mensaje”);
}
```

assertEquals - con margen de error para decimales

```
@Test
public void igualdadTest() {
    float  esperado = 3.1416f;
    float resultado = 3.14f;
    float delta = 2f;
    assertEquals(esperado, resultado, delta);
}
```

Una llamada a función en cada caso de prueba*

*salvo que estén *fuertemente* conectadas como en `DivisionLenta`